Problem Statement 1 (ProbID - P1): Robust Low-Light Image Restoration and Enhancement



Challenge Question

How can you design an intelligent, illumination-adaptive image restoration system that enhances severely underexposed, noisy and colour-distorted images while preserving fine textures, edges and structural information across different lighting conditions?

Problem Description

Develop an advanced low-light image restoration system capable of processing images captured under varying illumination conditions, including extremely dark environments, non-uniform lighting, sensor noise and colour distortion.

The system must go beyond basic brightness enhancement. It should intelligently estimate illumination, suppress noise, restore colour fidelity and recover structural details without introducing overexposure, halo artifacts or unnatural colours.

Mandatory technical requirements

A. Classical Computer Vision

- Illumination estimation and decomposition.
- Retinex-based enhancement.
- Adaptive histogram equalization (CLAHE).
- Edge-preserving denoising using bilateral or guided filtering.
- Contrast and colour correction.
- Adaptive parameter selection based on image characteristics.

B. Deep Learning

- Implement or adapt a low-light enhancement architecture such as RetinexNet, Zero-DCE, EnlightenGAN or another suitable model.
- Fine-tune or adapt the model wherever suitable training data is available.
- Demonstrate generalization across different illumination conditions.
- Include an image-quality-aware enhancement strategy.

C. Hybrid Intelligence

- Integrate classical illumination estimation with deep learning restoration.
- Develop an adaptive mechanism to select or combine enhancement outputs.
- Preserve edges and textures while minimizing noise amplification.
- Handle extremely dark and non-uniformly illuminated inputs.

Advanced challenge requirements

1. Develop an adaptive enhancement strategy instead of applying fixed enhancement parameters.
2. Compare at least three approaches: classical only, deep learning only and hybrid.
3. Perform ablation analysis to demonstrate the contribution of individual pipeline components.
4. Identify and minimize overexposure, colour shifts and detail loss.
5. Test the system on previously unseen low-light images.

Evaluation metrics

- PSNR and SSIM (when ground-truth references are available).
- LPIPS for perceptual quality.
- NIQE or another suitable no-reference quality metric.
- Processing latency.
- Noise suppression and edge preservation.

Expected Deliverable: A deployed intelligent low-light restoration application with adaptive enhancement, comparative evaluation, intermediate visualizations and documented failure analysis.

Problem Statement 2 (ProbID - P2): Intelligent and Computationally Efficient Smartphone ISP

Challenge Question

How can you develop a learned smartphone Image Signal Processing (ISP) pipeline that transforms degraded RAW sensor data into high-quality RGB images while optimizing colour accuracy, noise reduction, sharpness and computational efficiency under mobile-device resource constraints?

Problem Description

Develop an intelligent ISP system that simulates the processing pipeline of a modern smartphone camera.

The system must process RAW or appropriately supplied degraded sensor images and generate high-quality RGB outputs. It should intelligently handle sensor noise, colour distortion, varying illumination, white balance errors and image artifacts.

The solution must demonstrate an efficient hybrid ISP architecture rather than relying entirely on a conventional image-processing pipeline or an off-the-shelf neural network.

Mandatory technical requirements

A. Classical Computer Vision

- Bayer RAW preprocessing and demosaicing, when Bayer RAW input is supplied.
- White balance correction.
- Colour correction matrix.
- Denoising and edge-preserving filtering.
- Gamma correction and tone mapping.
- Image sharpening and colour-space transformations.

B. Deep Learning

- Implement or adapt a learned ISP architecture such as PyNET, U-Net-based ISP, or another suitable RAW-to-RGB model.
- Perform model adaptation or fine-tuning using an appropriate dataset.
- Learn image restoration and colour transformation.
- Demonstrate robustness to noise and illumination variations.

C. Hybrid ISP Architecture

- Integrate traditional ISP stages with learned image-processing components.
- Develop an optimized processing sequence.

- Avoid colour distortion and loss of fine details.
- Compare traditional ISP, learned ISP and hybrid ISP outputs.

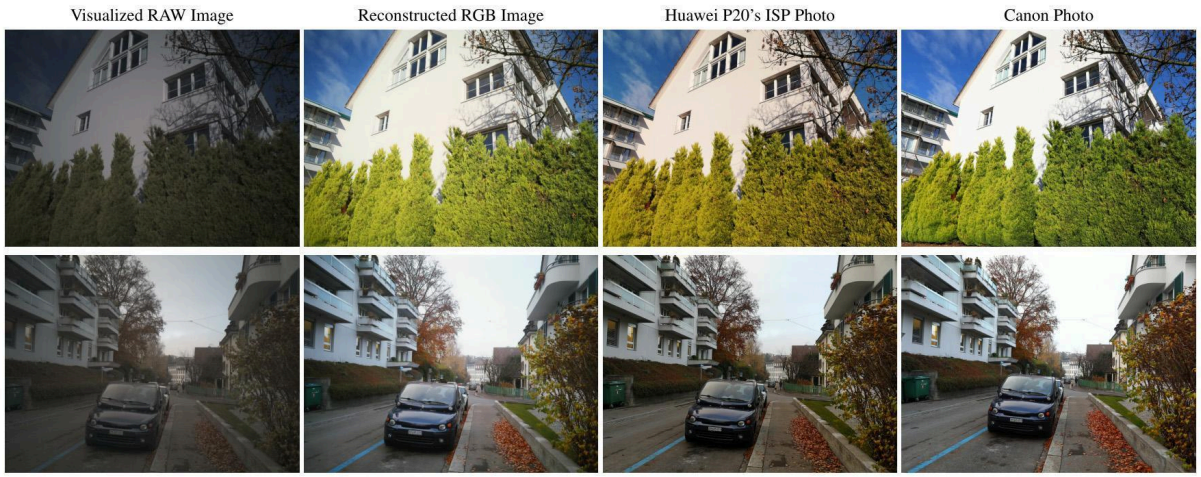
Advanced challenge requirements

1. Design an adaptive processing pipeline for varying sensor noise and illumination.
2. Demonstrate RAW-to-RGB reconstruction wherever RAW Bayer input is available.
3. Optimize the model using at least one technique: quantization, pruning, knowledge distillation or reduced precision.
4. Measure model size, memory requirements and inference latency.
5. Demonstrate performance degradation or recovery under challenging image conditions.
6. Compare output quality against a conventional ISP baseline.

Evaluation metrics

- PSNR and SSIM.
- Colour accuracy using Delta E.
- LPIPS or an appropriate perceptual metric.
- Model parameters and model size.
- Inference latency and memory consumption.

Expected Deliverable: A hybrid learned ISP application that demonstrates high-quality RAW image reconstruction, computational optimization and measurable improvements over conventional processing baselines.



Problem Statement 3 (ProbID - P3): Resource-Efficient Text-to-Image Generation for Mobile Devices

Challenge Question

How can you engineer a lightweight, high-quality text-to-image generation system that generates semantically accurate and visually realistic images from natural-language prompts while significantly reducing computational complexity, memory consumption and inference latency for resource-constrained mobile devices?

Problem Description

Develop an optimized text-to-image generation system that accepts complex natural-language descriptions and generates corresponding high-quality images.

The challenge is to reduce the computational requirements of generative models without significantly compromising semantic alignment, visual quality and image diversity.

The solution must demonstrate model optimization and efficient image generation, rather than simply integrating a publicly available text-to-image API.

Mandatory technical requirements

A. Classical Computer Vision

- Image resizing and resolution optimization.
- Colour correction and image enhancement.
- Edge-aware post-processing.
- Image quality assessment.
- Artifact detection and correction.

B. Deep Learning

- Implement or adapt a lightweight diffusion-based text-to-image architecture.
- Use a pretrained model with meaningful adaptation.
- Optimize the inference pipeline.
- Implement prompt encoding and conditional image generation.

C. Model Optimization

- Demonstrate at least two optimization strategies, such as:
    - Quantization or reduced-precision inference.
    - Pruning or knowledge distillation.
    - Reduced diffusion sampling steps.
    - Lightweight architecture selection.

    - Latent-space optimization.
- Compare optimized and baseline model performance.
- Demonstrate resource requirements for the intended mobile deployment target.

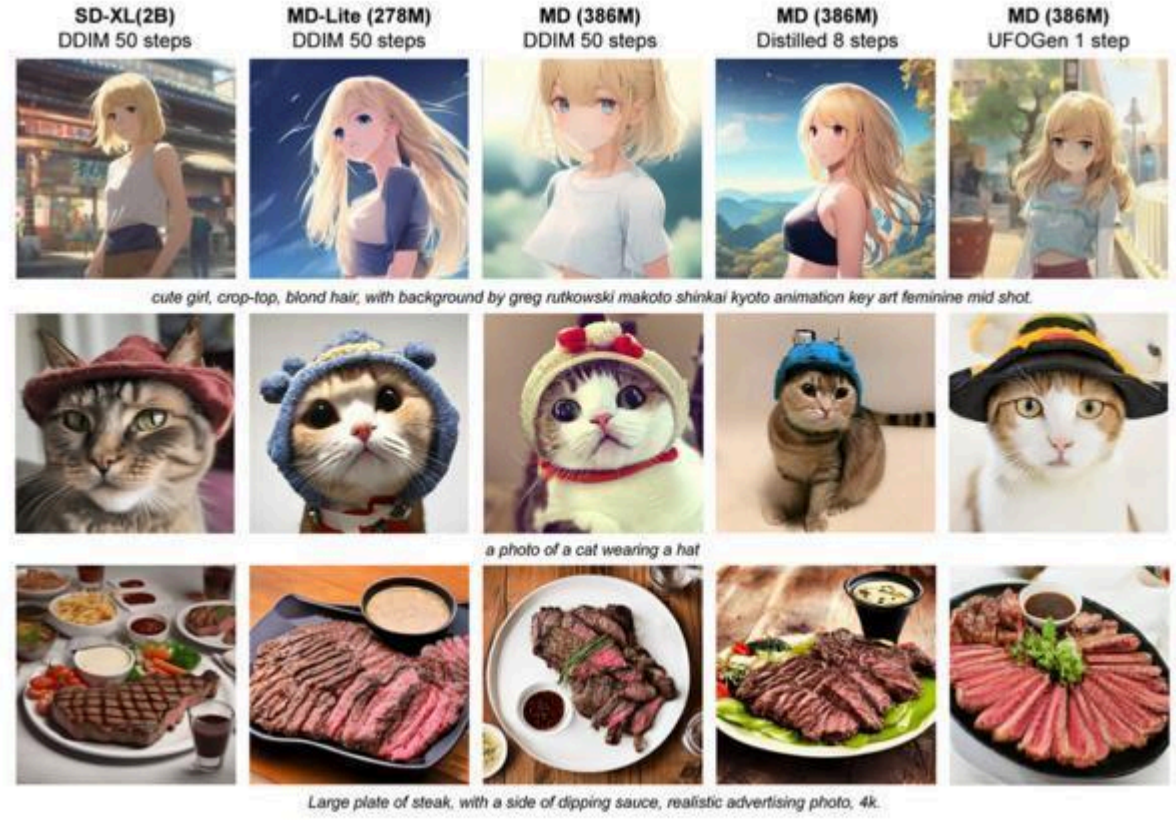
Advanced challenge requirements

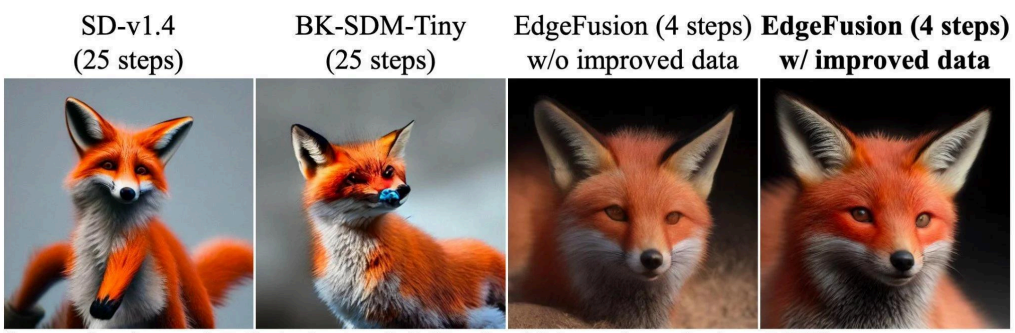
1. Generate images for complex prompts involving multiple objects, attributes and spatial relationships.
2. Evaluate semantic consistency between prompts and generated images.
3. Reduce inference latency and memory requirements.
4. Investigate quality degradation caused by model compression or fewer sampling steps.
5. Implement an adaptive generation strategy based on computational budget.
6. Demonstrate a fallback or resource-aware generation mechanism for constrained devices.

Evaluation metrics

- CLIP similarity for text-image alignment.
- FID or another suitable distribution-level metric, where an adequate evaluation dataset is available.
- Image generation latency.
- Peak memory consumption.
- Model size.
- Quality versus computational-cost comparison.

Expected Deliverable: A resource-efficient text-to-image generation application demonstrating optimized generative inference, prompt consistency and a quantitative comparison with the baseline model.





“a cute fox, hyper detailed, cinematic light, ultra high res, best shadow, RAW”



“a child sporting a yellow raincoat, vibrant against the gray of a rainy day, high resolution, 4k”

Problem Statement 4 (ProbID - P4): Edge-Preserving Depth Map Upsampling and Refinement

Challenge Question

How can you develop an intelligent, edge-aware depth reconstruction system that transforms sparse or low-resolution depth maps into dense, high-resolution depth representations while accurately preserving object boundaries, geometric structures and depth discontinuities?

Problem Description

Develop an advanced depth upsampling and refinement application that takes low-resolution or sparse depth measurements and reconstructs high-resolution depth maps.

The system must address challenges such as missing depth values, sensor noise, discontinuities between foreground and background objects, texture-related artifacts and inaccurate object boundaries.

Where RGB guidance images are provided, the system should exploit colour and structural information to improve depth reconstruction without introducing texture-copying artifacts.

Mandatory technical requirements

A. Classical Computer Vision

- Nearest-neighbour, bilinear and bicubic interpolation baselines.
- Joint bilateral filtering.
- Guided filtering.
- Edge detection and boundary-aware refinement.
- Depth discontinuity detection.
- Noise removal and hole filling.

B. Deep Learning

- Implement or adapt a depth completion or depth upsampling network.
- Suitable architectures may include guided depth super-resolution networks, CNN-based refinement models or transformer-based depth models.
- Fine-tune or adapt the model using suitable depth datasets.
- Incorporate RGB guidance where available.

C. Hybrid Depth Reconstruction

- Combine interpolation and edge-aware filtering with deep learning.
- Develop a depth refinement mechanism that preserves object boundaries.

- Handle sparse, noisy and incomplete depth maps.
- Prevent oversmoothing and texture-copying artifacts.

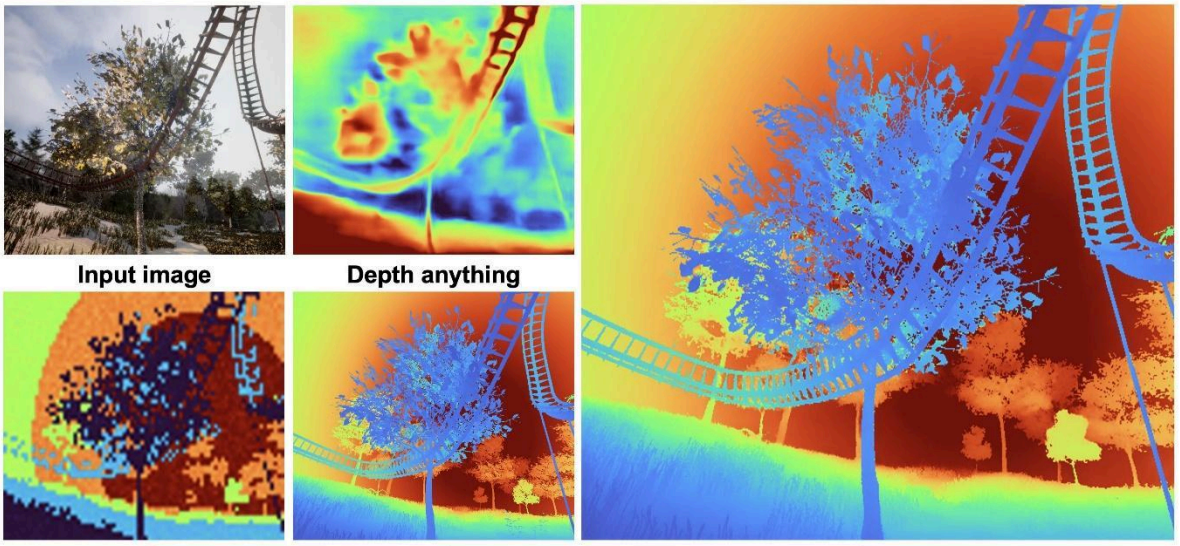
Advanced challenge requirements

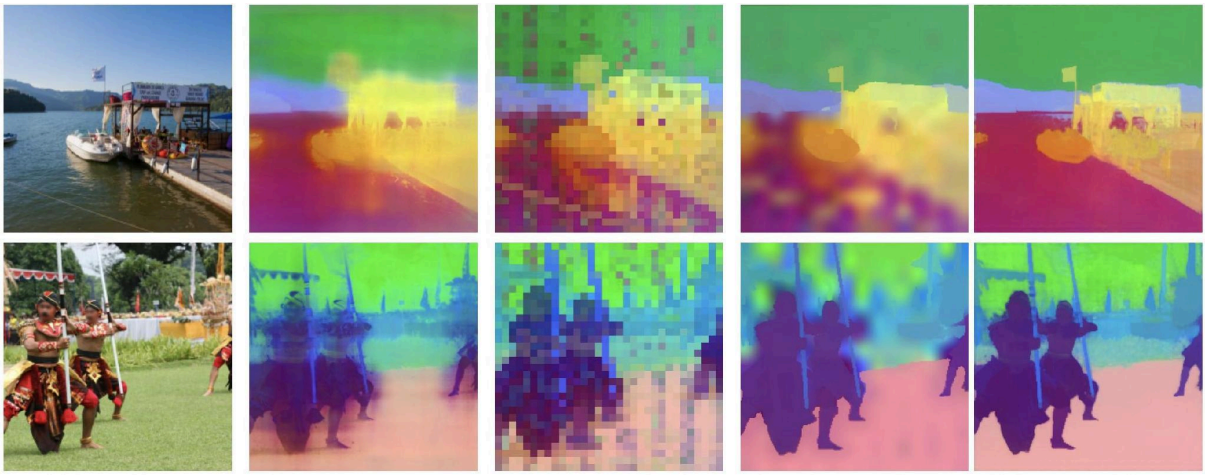
1. Support multiple upsampling factors, such as 2×, 4× and 8×.
2. Handle missing depth values and noisy depth measurements.
3. Demonstrate edge-aware depth reconstruction using RGB guidance.
4. Compare interpolation, classical refinement, deep learning and hybrid reconstruction.
5. Evaluate performance on unseen depth maps.
6. Demonstrate how the system performs around object boundaries and depth discontinuities.

Evaluation metrics

- RMSE and MAE for depth accuracy.
- PSNR and SSIM where appropriate.
- Edge preservation and boundary accuracy.
- Performance across different upsampling factors.
- Processing latency and memory consumption.

Expected Deliverable: A deployed intelligent depth reconstruction application that generates high-resolution depth maps and demonstrates improved depth accuracy and boundary preservation over conventional interpolation methods.





(a) Original image (b) Per-image optimized (c) 2x features (LiFT) (d) Mask-Bicubic (Stage 1) (e) Self-Distilled (Stage 2)

Problem Statement 5 (ProbID - P5): Intelligent Fine-Grained Vehicle Recognition and Analysis System

Challenge Question

How can you develop an intelligent vehicle recognition system that detects vehicles in real-world images and identifies their manufacturer (Make), Model and Manufacturing Year using a hybrid of classical computer vision and deep learning techniques, while handling visually similar vehicle categories, varying illumination, occlusion and different viewing angles?

1. Problem Description

Develop an intelligent vehicle identification application using the Stanford Cars Dataset.

The system should identify vehicles at a fine-grained classification level, distinguishing between visually similar car models, manufacturers and manufacturing years.

The application must localize vehicles using bounding boxes, classify the detected vehicle and display its predicted make, model, year and confidence score.

2. Dataset Description

Dataset: Stanford Cars Dataset

- Total images: 16,185
- Number of classes: 196
- Training images: 8,144
- Testing images: 8,041
- Annotations: Bounding-box regions.
- Classification: Car make, model and year.

3. Mandatory Technical Components

A. Classical Computer Vision

- Image preprocessing and normalization.
- Edge detection and contour analysis.
- Colour and texture feature extraction.
- Region-of-interest extraction.
- Geometric feature analysis.

B. Deep Learning

- CNN or Vision Transformer-based fine-grained classification.
- YOLO or R-CNN-based vehicle detection.

- Transfer learning and model fine-tuning.
- Multi-class vehicle classification across 196 categories.

C. Hybrid Integration

- Integrate classical image processing with deep learning.
- Use detected vehicle regions for fine-grained classification.
- Generate bounding boxes with predicted labels and confidence scores.
- Implement low-confidence prediction handling.

4. Advanced Challenge Requirements

1. Distinguish visually similar vehicle makes, models and manufacturing years.
2. Detect and classify vehicles under varying illumination and viewing angles.
3. Handle partial occlusion and different image resolutions.
4. Compare classical-feature-based, deep-learning-based and hybrid approaches.
5. Perform ablation analysis to demonstrate the contribution of individual components.
6. Apply at least one model optimization technique, such as quantization, pruning or reduced precision.
7. Evaluate the system using unseen test images.
8. Identify at least three failure cases and explain their causes.

5. Evaluation Metrics

Evaluation parameter	**Metric**
Classification performance	Top-1 and Top-5 accuracy
Object localization	Intersection over Union (IoU)
Object detection	Mean Average Precision (mAP)
Model efficiency	Model size and inference time
Robustness	Performance under varying conditions
Confidence	Prediction confidence and threshold analysis

6. Expected Application Output

The deployed application should:

1. Accept an uploaded vehicle image.

2. Detect the vehicle and draw a bounding box.
3. Predict the vehicle manufacturer, model and year.
4. Display classification confidence.
5. Show intermediate image-processing results.
6. Compare classical, deep learning and hybrid model performance.

Important implementation constraint

The Stanford Cars dataset supports fine-grained car classification and localization. It should not be assumed to provide number-plate recognition, vehicle tracking or traffic-event annotations. These should not be mandatory requirements for this challenge.

Expected outcome: A functional, intelligent vehicle identification system that combines object detection, fine-grained classification and classical computer vision, with measurable improvements in accuracy and computational efficiency.

Datasheet

Problem statement 1
Dataset links:

- LOL dataset
- LOL-v2 dataset
- Zero-DCE research implementation
- MIT-Adobe FiveK

Problem statement 2
Dataset links:

- SIDD dataset
- Modular Neural ISP datasets and code

Problem statement 3
Dataset links:

- MS COCO
- Flickr30K
- Hugging Face datasets

Problem statement 4
Dataset links:

- NYU Depth V2
- KITTI dataset
- Middlebury Stereo

Problem statement 5
Dataset Links:

- Official Stanford Cars Dataset

Instruction Sheet

Mandatory Deliverables

- Live hosted application URL.
- GitHub repository with
    - Complete source code.
    - System architecture diagram.
    - Design document.
    - Comparative evaluation results(an eval_probID_TeamID.md file with metrics in the proper order as in this document with first line having all the USNs comma seperated.). Example file name: *eval_P1_T01*
    - Ablation study and failure-case analysis.

Rubrics (100 points)

Demo and viva carry the most weight at 25 points, and each of the other five criteria carries 15.

Criterion	Max	Top band	Middle band	Low band
Evaluation Metrics	25	**12 to 15:** Average of Performance metrics among top 10% participants	**6 to 11:** bAverage of Performance metrics among top 60-90% participants	**0 to 5:** Average of Performance metrics among bottom 60% participants
Implementation	20	**12 to 15:** core features work on unseen inputs, a proper train/validation split, clean modular code, bad input handled	**6 to 11:** works on example inputs, some metrics, limited handling of bad input	**0 to 5:** partial flow, hard-coded or stubbed results, no evaluation
UI/UX and deployment	15	**12 to 15:** public URL works on first click from phone and laptop, responds within about 10 seconds, interface is clear with loading and error states, repo runs with one command or a Dockerfile, no secrets committed	**6 to 11:** works but is slow, flaky or hard to use	**0 to 5:** local only, or the URL is down at judging
Project documentation	15	**12 to 15:** architecture diagram matches the code, design doc has every template section, README reproduces the results,	**6 to 11:** present but sections are thin or the diagram is out of date	**0 to 5:** missing sections or no diagram

Criterion	Max	Top band	Middle band	Low band
		limitations and ethics stated honestly		
Demo and viva	25	**21 to 25:** clear 3-minute demo on the live URL that handles a judge's unseen input, and every member can explain their own part, the model and classical choices, the metrics and a failure case	**11 to 20:** demo works, but some members cannot explain their part or the design choices	**0 to 10:** demo fails, or answers show the team does not understand its own code

Deductions and tie-break

- Late submission: minus 5 points for every 15 minutes beyond deadline.
- Missing deliverable: no live URL caps UI/UX and deployment at 0, no GitHub link caps Implementation at 0, and no architecture diagram or design doc caps Project documentation at 5.
- Tie-break: higher Demo and viva score wins, then higher Implementation.